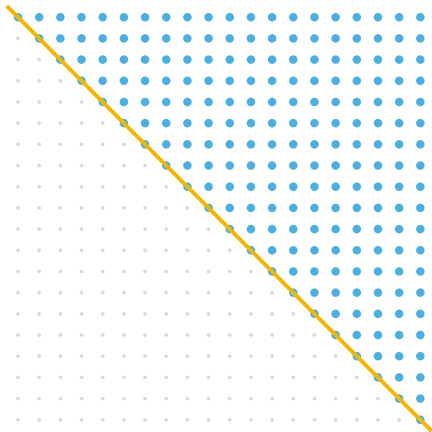


Soluciones Directas para Sistemas Lineales

Desde la aritmética del computador hasta la
factorización avanzada

Autor: Dr. Hermes Pantoja Carhuavilca

Curso: Métodos Numéricos



01

Implementar

Algoritmos de eliminación gaussiana y factorización LU para la resolución de sistemas matriciales de gran escala.

02

Evaluar

El número de condicionamiento $\text{cond}(A)$ para predecir la sensibilidad de la solución ante perturbaciones en los datos.

03

Validar

La estabilidad numérica mediante pivoteo parcial, minimizando el error de redondeo de la aritmética de punto flotante.

04

Analizar

Sistemas físicos reales (microrredes eléctricas, balances de flujo) traduciéndolos a modelos $A\mathbf{x} = \mathbf{b}$.

Anatomía del Sistema: Forma Algebraica y Matricial

Representación algebraica

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1 \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2 \\ \vdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m \end{cases}$$

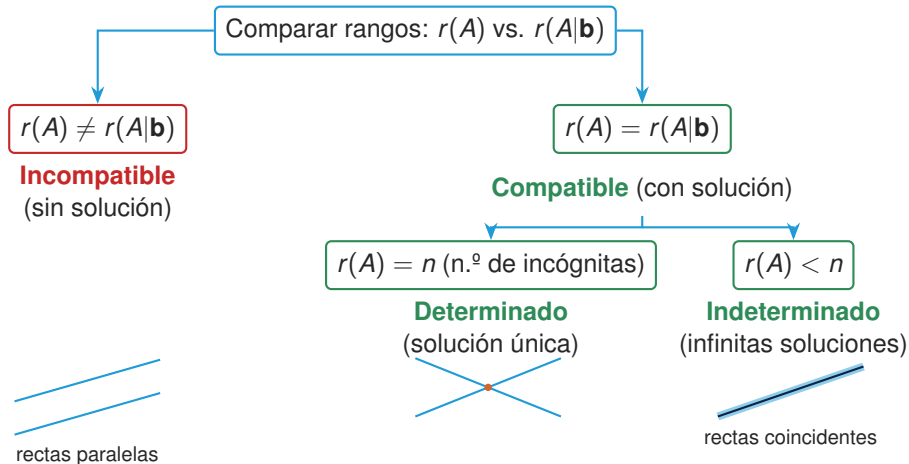
Representación matricial

$$\underbrace{\begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix}}_{A: m \times n} \underbrace{\begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}}_{\mathbf{x}: n \times 1} = \underbrace{\begin{bmatrix} b_1 \\ \vdots \\ b_m \end{bmatrix}}_{\mathbf{b}: m \times 1}$$

Matriz aumentada $[A | \mathbf{b}]$:

$$\left[\begin{array}{cccc|c} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{array} \right]$$

Existencia y Unicidad: Teorema de Rouché–Frobenius



Estrategia de Solución: Triangularización

Transformar $A\mathbf{x} = \mathbf{b}$ en $U\mathbf{x} = \mathbf{c}$ mediante operaciones elementales

Matriz completa

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}$$

Eliminación
gaussiana

Triangular superior

$$\begin{bmatrix} u_{11} & u_{12} & \cdots & u_{1n} \\ 0 & u_{22} & \cdots & u_{2n} \\ \vdots & \ddots & \ddots & \vdots \\ 0 & \cdots & 0 & u_{nn} \end{bmatrix}$$

Objetivo: eliminar incógnitas secuencialmente.

3 operaciones elementales permitidas

1. **Intercambio** ($R_i \leftrightarrow R_j$): crucial para el pivoteo.
2. **Escalamiento** ($R_i \rightarrow kR_i$, $k \neq 0$): multiplicar una fila por un escalar no nulo.
3. **Eliminación** ($R_i \rightarrow R_i - \lambda R_j$): sumar un múltiplo de una fila a otra.

Algoritmo Base: Eliminación Gaussiana (Simple)

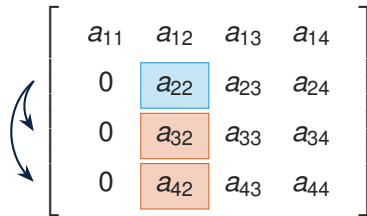
Fase 1: eliminación hacia adelante

Objetivo: hacer ceros debajo de la diagonal principal.

Multiplicador: $m_{ik} = \frac{a_{ik}}{a_{kk}}$

```
for k = 1:n-1
    for i = k+1:n
        m = A(i,k) / A(k,k);
        A(i,k:n) = A(i,k:n) - m*A(k,k:n);
        b(i) = b(i) - m*b(k);
    end
end
```

Etapa $k = 2$: pivote y elementos a anular


$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ 0 & a_{22} & a_{23} & a_{24} \\ 0 & a_{32} & a_{33} & a_{34} \\ 0 & a_{42} & a_{43} & a_{44} \end{bmatrix}$$

$$R_i \leftarrow R_i - m_{ik} R_k$$

Resolución: Sustitución Hacia Atrás

Fase 2: despeje recursivo

Se resuelve desde la última ecuación hacia la primera: $x_n = b_n / u_{nn}$ y

$$x_i = \frac{b_i - \sum_{j=i+1}^n u_{ij}x_j}{u_{ii}}$$
$$i = n - 1, n - 2, \dots, 1$$

Costo: eliminación $\approx \frac{2}{3}n^3 \Rightarrow O(n^3)$ (costoso);
sustitución $\approx n^2 \Rightarrow O(n^2)$ (rápido).

$$\begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ 0 & u_{22} & u_{23} & u_{24} \\ 0 & 0 & u_{33} & u_{34} \\ 0 & 0 & 0 & u_{44} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix}$$

orden de cálculo

Ejemplo: Eliminación Gaussiana Paso a Paso

$$\begin{array}{rcl} 2x_1 + x_2 - x_3 & = & 8 \\ -3x_1 - x_2 + 2x_3 & = & -11 \\ -2x_1 + x_2 + 2x_3 & = & -3 \end{array} \quad \Rightarrow \quad \left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ -3 & -1 & 2 & -11 \\ -2 & 1 & 2 & -3 \end{array} \right]$$

Etapla 1 ($m_{21} = -1.5$, $m_{31} = -1$):

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 2 & 1 & 5 \end{array} \right]$$

Etapla 2 ($m_{32} = 2/0.5 = 4$):

$$\left[\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & 0.5 & 0.5 & 1 \\ 0 & 0 & -1 & 1 \end{array} \right]$$

Sustitución hacia atrás:

$$x_3 = \frac{1}{-1} = -1$$

$$x_2 = \frac{1 - 0.5(-1)}{0.5} = 3$$

$$x_1 = \frac{8 - 3 - (-1)(-1)}{2} = 2$$

Solución: $\mathbf{x} = (2, 3, -1)^T$

Fragilidad Numérica: El Problema de los Pivotes Pequeños

El escenario de falla

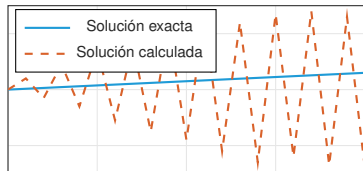
Pivote $a_{kk} \approx 0$ (p. ej. 0.00001) ▲

⇒ Multiplicador $m = \text{grande}/0.00001 = \text{enorme}$

⇒ El **error de redondeo** se **multiplica** por ese valor enorme.

Consecuencias

- **Amplificación de errores:** la precisión finita del computador destruye la solución.
- **División por cero:** si $a_{kk} = 0$, el algoritmo se detiene.



Eliminación Gaussiana: El Riesgo del Pivote Pequeño

Crítico: si $a_{kk} \approx 0$, entonces

$$m_{ik} = a_{ik}/a_{kk} \rightarrow \infty.$$

Ejemplo (solución exacta $x_1 = 1/3$, $x_2 = 2/3$):

$$0.0003 x_1 + 3.0000 x_2 = 2.0001$$

$$1.0000 x_1 + 1.0000 x_2 = 1.0000$$

Sin pivoteo, $m_{21} = 1/0.0003 \approx 3333$.

Luego

$$x_1 = \frac{2.0001 - 3x_2}{0.0003}$$

sufre **cancelación catastrófica**.

Cifras sig.	x_2	x_1 (sin pivoteo)	Error rel. x_1
3	0.667	-3.33	1099 %
4	0.6667	0.0000	100 %
5	0.66667	0.30000	10 %
6	0.666667	0.330000	1 %
7	0.6666667	0.3330000	0.1 %

Pivote pequeño: destruye la precisión si no se intercambian filas. (Datos: Chapra & Canale.)

Estrategias de Estabilidad: El Pivoteo

Definición: intercambio de filas para colocar el coeficiente de mayor magnitud en la posición del pivote.

Pivoteo parcial (estándar industrial):

1. En la etapa k , buscar p tal que $|a_{pk}| = \max_{k \leq i \leq n} |a_{ik}|$.
2. Intercambiar fila $k \leftrightarrow$ fila p .

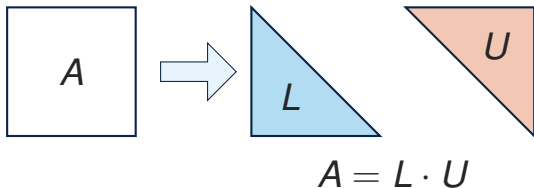
Ventaja: garantiza $|m_{ik}| \leq 1$, acotando el error.

Pivoteo completo: busca el máximo en filas y columnas (costoso y raramente usado).

$$\begin{bmatrix} 0.0003 & 3 \\ 1 & 1 \end{bmatrix} \xrightarrow{R_1 \leftrightarrow R_2} \begin{bmatrix} 1 & 1 \\ 0.0003 & 3 \end{bmatrix}$$

Ahora $m_{21} = 0.0003$ y, con solo 3 cifras, se obtiene $x_1 = 0.333$, $x_2 = 0.667$ ✓

Factorización LU: Eficiencia para Múltiples Escenarios



Flujo de trabajo:

1. Descomposición: $A \rightarrow L, U$ (costo $O(n^3)$, una sola vez)
2. Sustitución hacia adelante: $Ly = b$ (hallar y)
3. Sustitución hacia atrás: $Ux = y$ (hallar x)

Variantes

Doolittle: $l_{ij} = 1$
(unos en la diagonal de L)

Crout: $u_{ij} = 1$
(unos en la diagonal de U)

Cholesky: $A = LL^T$
(matrices simétricas
definidas positivas)

Variantes LU: Métodos de Doolittle y Crout

Método de Doolittle

$$L = \begin{bmatrix} \boxed{1} & & & \\ l_{21} & \boxed{1} & & \\ l_{31} & l_{32} & \boxed{1} & \\ l_{41} & l_{42} & l_{43} & \boxed{1} \end{bmatrix} \quad U = \begin{bmatrix} u_{11} & u_{12} & u_{13} & u_{14} \\ & u_{22} & u_{23} & u_{24} \\ & & u_{33} & u_{34} \\ & & & u_{44} \end{bmatrix}$$

$l_{ii} = 1$. Natural de la eliminación gaussiana:
 $l_{ik} = m_{ik}$ (los multiplicadores).

Método de Crout

$$L = \begin{bmatrix} l_{11} & & & \\ l_{21} & l_{22} & & \\ l_{31} & l_{32} & l_{33} & \\ l_{41} & l_{42} & l_{43} & l_{44} \end{bmatrix} \quad U = \begin{bmatrix} \boxed{1} & u_{12} & u_{13} & u_{14} \\ & \boxed{1} & u_{23} & u_{24} \\ & & \boxed{1} & u_{34} \\ & & & \boxed{1} \end{bmatrix}$$

$u_{ii} = 1$.

Ambos métodos requieren $\approx \frac{2}{3}n^3$ operaciones. La elección depende de la implementación.

Ejemplo: Factorización LU (Doolittle)

Con los multiplicadores del ejemplo anterior ($m_{21} = -1.5$, $m_{31} = -1$, $m_{32} = 4$):

$$\underbrace{\begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}}_A = \underbrace{\begin{bmatrix} 1 & 0 & 0 \\ -1.5 & 1 & 0 \\ -1 & 4 & 1 \end{bmatrix}}_L \underbrace{\begin{bmatrix} 2 & 1 & -1 \\ 0 & 0.5 & 0.5 \\ 0 & 0 & -1 \end{bmatrix}}_U$$

1. Adelante $Ly = \mathbf{b}$, con $\mathbf{b} = (8, -11, -3)^T$: **2. Atrás** $Ux = \mathbf{y}$:

$$y_1 = 8, \quad y_2 = -11 + 1.5(8) = 1, \quad y_3 = -3 + 8 - 4(1) = 1 \quad x_3 = -1, \quad x_2 = 3, \quad x_1 = 2$$

Si cambia $\mathbf{b}$, se reutilizan L y U : solo se repiten las dos sustituciones, de costo $O(n^2)$.

Factorización de Cholesky: Simetría y Velocidad

Requisitos

1. **Simétrica:** $A = A^T$ (espejo respecto a la diagonal).
2. **Definida positiva:** $\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$ para todo $\mathbf{x} \neq \mathbf{0}$ (energía positiva).

Ventajas

- **50 % de memoria** (solo el triángulo inferior).
- **50 % de operaciones** ($\approx n^3/3$).
- **Estable:** no requiere pivoteo.

$$A = L L^T$$

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} = \begin{bmatrix} l_{11} & 0 & 0 \\ l_{21} & l_{22} & 0 \\ l_{31} & l_{32} & l_{33} \end{bmatrix} \begin{bmatrix} l_{11} & l_{21} & l_{31} \\ 0 & l_{22} & l_{32} \\ 0 & 0 & l_{33} \end{bmatrix}$$

$$l_{kk} = \sqrt{a_{kk} - \sum_{j=1}^{k-1} l_{kj}^2}$$
$$l_{ik} = \frac{a_{ik} - \sum_{j=1}^{k-1} l_{ij} l_{kj}}{l_{kk}} \quad (i > k)$$

Ejemplo: Factorización de Cholesky

$$A = \begin{bmatrix} 4 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 4 \end{bmatrix}$$

$$l_{11} = \sqrt{4} = 2$$

$$l_{21} = -2/2 = -1, \quad l_{31} = 1/2 = 0.5$$

$$l_{22} = \sqrt{4 - (-1)^2} = \sqrt{3} \approx 1.7321$$

$$l_{32} = \frac{-2 - (0.5)(-1)}{\sqrt{3}} = -\frac{\sqrt{3}}{2} \approx -0.8660$$

$$l_{33} = \sqrt{4 - 0.25 - 0.75} = \sqrt{3} \approx 1.7321$$

$$L = \begin{bmatrix} 2 & 0 & 0 \\ -1 & 1.7321 & 0 \\ 0.5 & -0.8660 & 1.7321 \end{bmatrix}$$

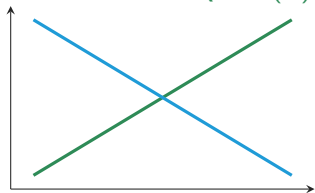
Todos los radicandos son positivos $\Rightarrow A$ es definida positiva. Si algún radicando resulta ≤ 0 , la matriz **no** es definida positiva.

Diagnóstico de Sistemas: Condicionamiento y Sensibilidad

Número de condicionamiento:

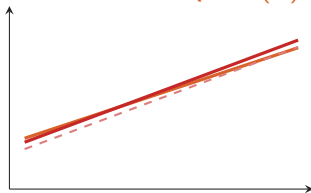
$$\text{cond}(A) = \|A\| \cdot \|A^{-1}\|$$

Bien condicionado ($\text{cond}(A) \approx 1$)



Pequeño error en datos $\rightarrow$ pequeño error en la solución.

Mal condicionado ($\text{cond}(A) \gg 1$)



Pequeña perturbación en datos $\rightarrow$ cambio catastrófico en la solución.

Ejemplo: $x + 2y = 3$, $1.0001x + 2y = 3.0001 \Rightarrow (1, 1)$. Si el segundo término cambia a 3.0002 $\Rightarrow (2, 0.5)$. Aquí $\text{cond}_{\infty}(A) \approx 6 \times 10^4$.

Aplicación en Ingeniería de Energía: Circuitos Resistivos

Contexto: microrred eléctrica

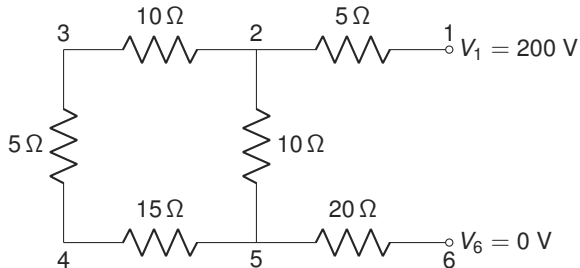
Principios físicos

1. Ley de corrientes de Kirchhoff (LCK):

$$\sum I_{\text{entrada}} = \sum I_{\text{salida}}$$

2. Ley de voltajes de Kirchhoff (LVK):

$$\sum V = 0 \text{ en una malla cerrada}$$



Modelado Matricial y Justificación de la Factorización LU

LCK en los nodos 2–5 (multiplicando por 60):

$$\underbrace{\begin{bmatrix} 24 & -6 & 0 & -6 \\ -6 & 18 & -12 & 0 \\ 0 & -12 & 16 & -4 \\ -6 & 0 & -4 & 13 \end{bmatrix}}_{\text{conductancias (fija)}} \underbrace{\begin{bmatrix} V_2 \\ V_3 \\ V_4 \\ V_5 \end{bmatrix}}_{\text{fuentes (variable)}} = \underbrace{\begin{bmatrix} 2400 \\ 0 \\ 0 \\ 0 \end{bmatrix}}_{\text{fuentes (variable)}}$$

Solución: $V_2 = 169.23$, $V_3 = 153.85$, $V_4 = 146.15$,
 $V_5 = 123.08$ V.

Matriz simétrica y definida positiva $\Rightarrow$ ¡apta para

Cholesky!

¿Por qué LU?

- En una red real, la **topología** (matriz) es constante, pero la **demanda** (vector) cambia minuto a minuto.
- Se factoriza $A = LU$ **una sola vez**.
- Para cada nuevo vector solo se resuelven sustituciones $O(n^2)$ en lugar de Gauss completo $O(n^3)$.
- Permite **simulación en tiempo real**.

Resumen Comparativo de Métodos Directos

Método	Características	Costo	Mejor uso
Gauss simple	Didáctico, inestable	$O(n^3)$	Aprendizaje académico
Gauss + pivoteo ✓ estándar	Robusto, estándar	$O(n^3)$	Sistemas generales $A\mathbf{x} = \mathbf{b}$
Factorización LU	Desacopla A de $\mathbf{b}$	$O(n^3)$ + sustituciones $O(n^2)$	Múltiples vectores de carga $\mathbf{b}$
Cholesky ✓ estándar	Requiere simetría y definida positiva	$\approx n^3/3$ (muy rápido)	Elementos finitos, matrices de rigidez

Actividad Práctica: El Copiloto Digital

El reto

- **Tarea:** optimización de scripts y diagnóstico.

1. Prompt inicial:

solicitar a ChatGPT/Claude un script MATLAB para resolver un sistema 10×10 usando Gauss simple.

- ### 2. Modificación:
- solicitar la inclusión de *pivoteo parcial* para manejar ceros.

```
% Gauss simple 10x10 (sin funciones integradas)
A = rand(10,10); b = rand(10,1);
n = length(b);
Aug = [A, b]; % Matriz aumentada
% Eliminacion hacia adelante
for k = 1:n-1
    for i = k+1:n
        m = Aug(i,k) / Aug(k,k);
        Aug(i,k:n+1) = Aug(i,k:n+1) ...
            - m*Aug(k,k:n+1);
    end
end
% Sustitucion hacia atras
x = zeros(n,1);
x(n) = Aug(n,n+1) / Aug(n,n);
for i = n-1:-1:1
    x(i) = (Aug(i,n+1) - Aug(i,i+1:n)*x(i+1:n)) ...
        / Aug(i,i);
end
disp('Solucion x:'); disp(x);
```

Análisis Crítico: Humano vs. Máquina

Checklist de validación técnica

▲ ¿Omitió la sustitución hacia atrás?

Las IA frecuentemente triangularizan la matriz pero olvidan calcular el vector $\mathbf{x}$ final.

✓ Manejo de índices

Verificar si el bucle maneja correctamente los índices de MATLAB (base 1).

▲ Validación de estabilidad

¿El código verifica $\text{cond}(A)$? Si $\text{cond}(A)$ es alto ($\sim 10^{14}$), el código fallará silenciosamente.

Lección: la IA escribe la sintaxis; el ingeniero garantiza la integridad numérica.

Logros Alcanzados

- ✓ Representación algebraica y matricial de SEL.
- ✓ Clasificación de soluciones (Rouché–Frobenius).
- ✓ Implementación de eliminación gaussiana y sustitución.
- ✓ Aplicación de pivoteo parcial para estabilidad.
- ✓ Factorización avanzada (LU y Cholesky).
- ✓ Diagnóstico de condicionamiento.

*“La aritmética exacta es una ilusión;
la estabilidad numérica es la meta.”*

Referencias Bibliográficas

1. Chapra, S. C., & Canale, R. P. *Métodos Numéricos para Ingenieros*. McGraw-Hill Education.
2. Burden, R. L., & Faires, J. D. *Análisis Numérico*. Cengage Learning.
3. Nakamura, S. *Métodos Numéricos Aplicados con Software*. Prentice Hall.